

INDUSTRIAL PROTOCOLS LAB #3

Name: Ali Irfan

Roll no: 22FA-036-CE

Codes:

- **axi_lite_master**

```
module axi_lite_master (
    input wire ACLK,
    input wire ARESETN,
    input wire START_READ,
    input wire START_WRITE,
    input wire [31:0] address,
    input wire [31:0] W_data,
    output reg [31:0] ARADDR,
    output reg ARVALID,
    input wire ARREADY,
    input wire [31:0] RDATA,
    input wire [1:0] RRESP,
    input wire RVALID,
    output reg RREADY,
    output reg [31:0] AWADDR,
    output reg AWVALID,
    input wire AWREADY,
    output reg [31:0] WDATA,
    output reg [3:0]WSTRB,
    output reg WVALID,
    input wire WREADY,
    input wire [1:0] BRESP,
    input wire BVALID,
    output reg BREADY
);

parameter IDLE = 3'b000;
parameter RADDR_CHANNEL = 3'b001;
parameter RDATA_CHANNEL = 3'b010;
parameter WRITE_CHANNEL = 3'b011;
parameter WRESP_CHANNEL = 3'b100;

reg [2:0] state, next_state;
reg [31:0] read_buf;

always @(posedge ACLK or negedge ARESETN) begin
    if (!ARESETN)
        state <= IDLE;
    else
        state <= next_state;
    end

always @(*) begin
    next_state = state;
    case (state)
```

```

IDLE: begin
if (START_READ)
next_state = RADDR_CHANNEL;
else if (START_WRITE)
next_state = WRITE_CHANNEL;
else
next_state = IDLE;
end
RADDR_CHANNEL: begin
if (ARVALID && ARREADY)
next_state = RDATA_CHANNEL;
end
RDATA_CHANNEL: begin
if (RVALID && RREADY)
next_state = IDLE;
end
WRITE_CHANNEL: begin
if (AWVALID && AWREADY && WVALID && WREADY)
next_state = WRESP_CHANNEL;
end
WRESP_CHANNEL: begin
if (BVALID && BREADY)
next_state = IDLE;
end
default: next_state = IDLE;
endcase
end

```

```

always @(posedge ACLK or negedge ARESETN) begin
if (!ARESETN) begin
ARADDR <= 0;
ARVALID <= 0;
RREADY <= 0;
AWADDR <= 0;
AWVALID <= 0;
WDATA <= 0;
WSTRB <= 0;
WVALID <= 0;
BREADY <= 0;
read_buf <= 0;
end else begin
case (state)
IDLE: begin
ARVALID <= 0;
RREADY <= 0;
AWVALID <= 0;
WVALID <= 0;
BREADY <= 0;
if (START_READ) begin
ARADDR <= address;
ARVALID <= 1;
end else if (START_WRITE) begin
AWADDR <= address;
AWVALID <= 1;

```

```

WDATA <= W_data;
WSTRB <= 4'b1111;
WVALID <= 1;
end
End
RADDR_CHANNEL: begin
if (ARVALID && ARREADY) begin
ARVALID <= 0;
RREADY <= 1;
end
end
RDATA_CHANNEL: begin
if (RVALID && RREADY) begin
read_buf <= RDATA;
RREADY <= 0;
end
end
WRITE_CHANNEL: begin
if (AWVALID && AWREADY)
AWVALID <= 0;
if (WVALID && WREADY)
WVALID <= 0;
if (AWVALID && AWREADY && WVALID && WREADY)
BREADY <= 1;
end
WRESP_CHANNEL: begin
if (BVALID && BREADY)
BREADY <= 0;
end
default: begin
ARVALID <= 0;
RREADY <= 0;
AWVALID <= 0;
WVALID <= 0;
BREADY <= 0;
end
endcase
end
end

Endmodule

```

- **axi_lite_slave**

```
module axi_lite_slave (  
    input wire ACLK,  
    input wire ARESETN,  
    input wire [31:0] ARADDR,  
    input wire ARVALID,  
    output reg ARREADY,  
    output reg [31:0] RDATA,  
    output reg [1:0] RRESP,  
    output reg RVALID,  
    input wire RREADY,  
    input wire [31:0] AWADDR,  
    input wire AWVALID,  
    output reg AWREADY,  
    input wire [31:0] WDATA,  
    input wire [3:0]WSTRB,  
    input wire WVALID,  
    output reg WREADY,  
    output reg [1:0] BRESP,  
    output reg BVALID,  
    input wire BREADY  
);  
  
reg [31:0] memory [0:255];  
  
parameter IDLE = 3'b000;  
parameter RADDR_CHANNEL = 3'b001;  
parameter RDATA_CHANNEL = 3'b010;  
parameter WRITE_CHANNEL = 3'b011;  
parameter WRESP_CHANNEL = 3'b100;  
  
reg [2:0] state, next_state;  
reg [31:0] raddr_reg;  
reg [31:0] waddr_reg;  
  
always @(posedge ACLK or negedge ARESETN) begin  
    if (!ARESETN)  
        state <= IDLE;  
    else  
        state <= next_state;  
    end  
  
    always @(*) begin  
        next_state = state;  
        case (state)  
            IDLE: begin  
                if (AWVALID && WVALID)  
                    next_state = WRITE_CHANNEL;  
                else if (ARVALID)  
                    next_state = RADDR_CHANNEL;  
                else  
                    next_state = IDLE;  
            end  
            RADDR_CHANNEL: begin
```

```

if (ARVALID && ARREADY)
next_state = RDATA_CHANNEL;
end
RDATA_CHANNEL: begin
if (RVALID && RREADY)
next_state = IDLE;
end
WRITE_CHANNEL: begin
if (AWVALID && AWREADY && WVALID && WREADY)
next_state = WRESP_CHANNEL;
end
WRESP_CHANNEL: begin
if (BVALID && BREADY)
next_state = IDLE;
end
default: next_state = IDLE;
endcase
end

```

```

integer k;
always @(posedge ACLK or negedge ARESETN) begin
if (!ARESETN) begin
ARREADY <= 0;
RDATA <= 0;
RRESP <= 2'b00;
RVALID <= 0;
AWREADY <= 0;
WREADY <= 0;
BRESP <= 2'b00;
BVALID <= 0;
raddr_reg <= 0;
waddr_reg <= 0;
for (k = 0; k < 256; k = k + 1)
memory[k] <= 0;
end else begin
case (state)
IDLE: begin
RVALID <= 0;
BVALID <= 0;
ARREADY <= 0;
AWREADY <= 0;
WREADY <= 0;
if (AWVALID && WVALID) begin
AWREADY <= 1;
WREADY <= 1;
waddr_reg <= AWADDR;
end else if (ARVALID) begin
ARREADY <= 1;
raddr_reg <= ARADDR;
end
end
RADDR_CHANNEL: begin
if (ARVALID && ARREADY) begin
ARREADY <= 0;

```

```

RVALID <= 1;
RRESP <= 2'b00;
RDATA <= memory[raddr_reg[7:0]];
end
end
RDATA_CHANNEL: begin
if (RVALID && RREADY) begin
RVALID <= 0;
end
end
WRITE_CHANNEL: begin
if (AWVALID && AWREADY && WVALID && WREADY) begin
AWREADY <= 0;
WREADY <= 0;
if (WSTRB[0]) memory[waddr_reg[7:0]][7:0] <= WDATA[7:0];
if (WSTRB[1]) memory[waddr_reg[7:0]][15:8] <= WDATA[15:8];
if (WSTRB[2]) memory[waddr_reg[7:0]][23:16] <= WDATA[23:16];
if (WSTRB[3]) memory[waddr_reg[7:0]][31:24] <= WDATA[31:24];
BVALID <= 1;
BRESP <= 2'b00;
end
end
WRESP_CHANNEL: begin
if (BVALID && BREADY) begin
BVALID <= 0;
end
end
default: begin
ARREADY <= 0;
RVALID <= 0;
AWREADY <= 0;
WREADY <= 0;
BVALID <= 0;
end
endcase
end
end
endmodule

```

- **axi_lite_top**

```
module axi_lite_top (  
  input wire clk,  
  input wire rst_n,  
  input wire start_read,  
  input wire start_write,  
  input wire [31:0] addr,  
  input wire [31:0] wr_data,  
  output wire [31:0] rd_data  
);
```

```
  wire [31:0] araddr_w;  
  wire arvalid_w;  
  wire arready_w;  
  wire [31:0] rdata_w;  
  wire [1:0] rresp_w;  
  wire rvalid_w;  
  wire rready_w;  
  wire [31:0] awaddr_w;  
  wire awvalid_w;  
  wire awready_w;  
  wire [31:0] wdata_w;  
  wire [3:0] wstrb_w;  
  wire wvalid_w;  
  wire wready_w;  
  wire [1:0] bresp_w;  
  wire bvalid_w;  
  wire bready_w;
```

```
  axi_lite_master u_master (  
    .ACLK(clk),  
    .ARESETN(rst_n),  
    .START_READ(start_read),  
    .START_WRITE(start_write),  
    .address(addr),  
    .W_data(wr_data),  
    .ARADDR(araddr_w),  
    .ARVALID(arvalid_w),  
    .ARREADY(arready_w),  
    .RDATA(rdata_w),  
    .RRESP(rresp_w),  
    .RVALID(rvalid_w),  
    .RREADY(rready_w),  
    .AWADDR(awaddr_w),  
    .AWVALID(awvalid_w),  
    .AWREADY(awready_w),  
    .WDATA(wdata_w),  
    .WSTRB(wstrb_w),  
    .WVALID(wvalid_w),  
    .WREADY(wready_w),  
    .BRESP(bresp_w),  
    .BVALID(bvalid_w),  
    .BREADY(bready_w)  
  );
```

```
axi_lite_slave u_slave (  
  .ACLK(clk),  
  .ARESETN(rst_n),  
  .ARADDR(araddr_w),  
  .ARVALID(arvalid_w),  
  .ARREADY(arready_w),  
  .RDATA(rdata_w),  
  .RRESP(rresp_w),  
  .RVALID(rvalid_w),  
  .RREADY(rready_w),  
  .AWADDR(awaddr_w),  
  .AWVALID(awvalid_w),  
  .AWREADY(awready_w),  
  .WDATA(wdata_w),  
  .WSTRB(wstrb_w),  
  .WVALID(wvalid_w),  
  .WREADY(wready_w),  
  .BRESP(bresp_w),  
  .BVALID(bvalid_w),  
  .BREADY(bready_w)  
);
```

```
assign rd_data = rdata_w;
```

```
endmodule
```


- **tb_axi_lite_top**

```
`timescale 1ns / 1ps

module tb_axi_lite_top;

    reg clk;
    reg rst_n;
    reg start_read;
    reg start_write;
    reg [31:0] addr;
    reg [31:0] wr_data;
    wire [31:0] rd_data;

    axi_lite_top uut (
        .clk(clk),
        .rst_n(rst_n),
        .start_read(start_read),
        .start_write(start_write),
        .addr(addr),
        .wr_data(wr_data),
        .rd_data(rd_data)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst_n = 0;
        start_read = 0;
        start_write = 0;
        addr = 0;
        wr_data = 0;

        #20;
        rst_n = 1;

        #20;
        @(posedge clk);
        addr = 32'h00000004;
        wr_data = 32'h5A5AA5A5;
        start_write = 1;
        @(posedge clk);
        start_write = 0;

        #100;
        @(posedge clk);
        addr = 32'h00000008;
        wr_data = 32'hDEADBEEF;
        start_write = 1;
        @(posedge clk);
        start_write = 0;

        #100;
        @(posedge clk);
```

```

addr = 32'h00000004;
start_read = 1;
@(posedge clk);
start_read = 0;

#100;
@(posedge clk);
addr = 32'h00000008;
start_read = 1;
@(posedge clk);
start_read = 0;

#100;
$finish;
end

Endmodule

```

• QuestaSim Waveform:

